

1. Operator Overloading — Core Idea

Operator overloading lets you define how an operator behaves for your own class.

For example:

```
class Complex {
    int real, imag;

public:
    Complex(int r, int i) : real(r), imag(i) {}

    Complex operator+(const Complex& other) {
        return Complex(real + other.real, imag + other.imag);
    }
};
```

Now:

```
Complex a(2, 3);
Complex b(4, 5);

Complex c = a + b;
```

The compiler essentially interprets:

```
a + b
```

as:

```
a.operator+(b)
```

for a member overload.

2. Rules for Operator Overloading

These are **interview-important rules**.

Rule 1: At least one operand must be a user-defined type

You cannot redefine operators for only built-in types.

✗ Invalid:

```
int operator+(int a, int b);
```

Why?

Because C++ doesn't allow you to change the meaning of operators for fundamental types.

But this is valid:

```
class Number {};
```

```
Number operator+(Number a, Number b);
```

Rule 2: You cannot create a new operator

You can overload existing operators only.

✗ Can't create:

```
operator**
```

or:

```
operator@
```

unless that operator already exists in C++.

Rule 3: You cannot change precedence

Suppose:

```
a + b * c
```

***** will still have higher precedence than **+**.

You cannot make your overloaded `+` execute first.

Rule 4: You cannot change associativity

For example:

```
a - b - c
```

remains:

```
(a - b) - c
```

You can't change it to:

```
a - (b - c)
```

through operator overloading.

Rule 5: You cannot change the number of operands

Unary operators remain unary.

Binary operators remain binary.

For example, `+` can have:

```
+a      // unary
a + b    // binary
```

but you cannot invent a three-operand version:

```
a + b + c    // ✗ not one 3-operand operator
```

3. Operators That CANNOT Be Overloaded

Memorize these:

OperatorMeaning`::`Scope resolution`.`Member access`.*`Pointer-to-member access`?:`Ternary/conditional`sizeof`Size of type/object

So:

```
:: . .* ?: sizeof
```

cannot be overloaded.

Interview trick

Don't confuse these with operators that **can** be overloaded.

For example:

```
new
delete
new[]
delete[]
```

can be overloaded.

And:

```
()
[]
->
```

can also be overloaded.

4. Member vs Non-Member Operator Overloading

An operator can often be implemented either as a member function or a non-member function.

Example:

```
class Number {
public:
    Number operator+(const Number& other);
};
```

Then:

```
a + b
```

becomes:

```
a.operator+(b);
```

A non-member version:

```
Number operator+(const Number& a, const Number& b);
```

Then:

```
a + b
```

becomes:

```
operator+(a, b);
```

This distinction becomes **very important** for `<<` and `>>`.

5. Why `<<` and `>>` Are Usually Friend Functions

Consider:

```
cout << obj;
```

The left operand is:

```
cout
```

whose type is essentially `ostream`.

If we tried to make `operator<<` a member of our class:

```
class Student {  
public:  
    ostream& operator<<(ostream& os);  
};
```

the syntax would require:

```
obj << cout;
```

because member operators work like:

```
obj.operator<<(cout);
```

But we want:

```
cout << obj;
```

The left operand is `cout`, not `Student`.

Therefore, `operator<<` is generally implemented as a **non-member function**:

```
class Student {  
    int age;  
  
public:  
    Student(int a) : age(a) {}  
  
    friend ostream& operator<<(ostream& os, const Student& s);  
};  
  
ostream& operator<<(ostream& os, const Student& s) {  
    os << s.age;  
    return os;  
}
```

Now:

```
Student s(20);
```

```
cout << s;
```

works.

Conceptually:

```
operator<<(cout, s);
```

Why `friend`?

Because `age` might be private.

```
class Student {  
private:  
    int age;  
};
```

The non-member function normally cannot access `age`.

Making it a friend gives it access:

```
friend ostream& operator<<(ostream&, const Student&);
```

So the usual pattern is:

```
friend ostream& operator<<(ostream&, const Class&);  
friend istream& operator>>(istream&, Class&);
```

6. Why Not Make `<<` a Member of `ostream`?

Technically, `ostream` already has its own `operator<<` overloads for built-in types and standard types.

But your custom class is yours to define behavior for.

You generally **don't modify** `std::ostream`.

Instead, you provide:

```
operator<<(ostream&, YourClass&)
```

as a non-member.

This is also why the left operand can remain `cout`.

7. Why Return `ostream&`?

This enables chaining:

```
cout << a << b << c;
```

Think of it as:

```
((cout << a) << b) << c;
```

If:

```
operator<<(cout, a)
```

returns `cout` as an `ostream&`, then the next `<< b` can operate on it.

Typical implementation:

```
ostream& operator<<(ostream& os, const Student& s) {  
    os << s.age;  
    return os;  
}
```

Same idea for input:

```
istream& operator>>(istream& is, Student& s) {  
    is >> s.age;  
    return is;  
}
```

8. What Is a Friend Function?

A **friend function** is a non-member function that has permission to access the private and protected members of a class.

Example:

```
class Box {
private:
    int width;

public:
    Box(int w) : width(w) {}

    friend void print(Box b);
};

void print(Box b) {
    cout << b.width; // allowed
}
```

Notice:

```
print()
```

is **not** a member function of `Box`.

But because it is declared:

```
friend void print(Box b);
```

it can access:

```
b.width
```

even though `width` is private.

9. Friend Function vs Member Function

This is a good interview comparison.

Member Function	Friend Function	Belongs to class?	Yes	No	Has <code>this</code> pointer?	Yes	No	Can access private members?	Yes	Yes	Called using object?	Usually <code>obj.foo()</code>	Usually <code>foo(obj)</code>	Inherited?	Member functions participate in inheritance	No	Can be used for symmetric operators?	Sometimes awkward	Often useful
-----------------	-----------------	-------------------	-----	----	--------------------------------	-----	----	-----------------------------	-----	-----	----------------------	--------------------------------	-------------------------------	------------	---	----	--------------------------------------	-------------------	--------------

10. What Is a Friend Class?

You can make an **entire class** a friend.

```
class A {
private:
    int x = 10;

    friend class B;
};

class B {
public:
    void print(A& a) {
        cout << a.x;
    }
};
```

Now **all member functions of `B`** can access private/protected members of `A`.

Important:

```
friendship is NOT mutual
friendship is NOT inherited
friendship is NOT transitive
```

Not mutual

If:

```
A is friend of B
```

that does NOT mean:

```
B is friend of A
```

Not transitive

If:

```
A is friend of B
B is friend of C
```

it does NOT mean:

```
A is friend of C
```

Not inherited

If:

```
class B : public A
```

and `A` has a friend `F`, that doesn't automatically make `F` a friend of `B`.

11. Does `friend` Break Encapsulation?

This is a **very common** interview question.

Short answer:

No, not necessarily.

Friendship intentionally provides **controlled access** to private members.

Encapsulation means:

```
Control access to an object's internal state.
```

A friend declaration is itself part of the class definition, so the class explicitly decides who gets access.

For example:

```
class BankAccount {
private:
    double balance;

    friend void audit(const BankAccount&);
};
```

Only `audit()` gets special access.

So `friend` doesn't automatically destroy encapsulation.

But...

Overusing friendship can **weaken encapsulation** because external code gets direct access to implementation details.

So interview answer:

```
"Friendship doesn't inherently violate encapsulation because the class explicitly controls which functions or classes get privileged access. However, excessive use of friend can increase coupling and expose implementation details, so it should be used judiciously."
```

That's a strong interview answer.

12. Important Operator Overloading Interview Question

Q: Which operators must be non-member functions?

The classic ones are:

```
=
[]
()
->
```

These **must be member functions**.

Also:

```
operator new
operator delete
operator new[]
operator delete[]
```

have special rules.

For normal interview purposes, remember:

```
=, [], (), and -> must be overloaded as member functions.
```


`<<` and `>>` do not have to be friends. They are typically non-member functions, and are often made `friend` when they need access to private data.

That's an important distinction.

13. Why Should `==` Often Be Non-Member?

Consider:

```
a == b
```

If implemented as member:

```
a.operator==(b)
```

This works.

But symmetric operators are often better as non-members:

```
bool operator==(const A& lhs, const A& rhs);
```

because both operands are treated symmetrically.

This becomes especially useful when implicit conversions are involved.

For example:

```
class Number {  
public:  
    Number(int x);  
};
```

A non-member operator can potentially allow conversions on **both operands**, whereas a member operator has a fixed left-hand operand type.

This is a more advanced interview point.

14. `friend` + `operator<<` — Complete Example

This is worth knowing almost by heart:

```
#include <iostream>  
using namespace std;  
  
class Student {  
private:  
    string name;  
    int age;  
  
public:  
    Student(string n, int a) : name(n), age(a) {}  
  
    friend ostream& operator<<(ostream& os, const Student& s);  
    friend istream& operator>>(istream& is, Student& s);  
};  
  
ostream& operator<<(ostream& os, const Student& s) {  
    os << s.name << " " << s.age;  
    return os;  
}  
  
istream& operator>>(istream& is, Student& s) {  
    is >> s.name >> s.age;  
    return is;  
}
```

Usage:

```
Student s("Lalit", 23);  
  
cout << s;
```

Internally:

```
operator<<(cout, s);
```

And:

```
cin >> s;
```

becomes:

```
operator>>(cin, s);
```

Interview Questions You Should Be Ready For

Basic

Q1. What is operator overloading?

Giving existing C++ operators a user-defined behavior for user-defined types.

Q2. Can we overload any operator?

No. These cannot be overloaded:

```
:: . .* ?: sizeof
```

Q3. Can we change operator precedence?

No.

Q4. Can we change associativity?

No.

Q5. Can we overload an operator for two `int`s?

No. At least one operand must be a user-defined type.

Medium

Q6. Why is `operator<<` usually a friend?

Because `cout << obj` has `ostream` as the left operand, so making it a member of `obj` would not work naturally. A non-member function handles both operands, and declaring it `friend` allows access to private members.

Q7. Does `operator<<` have to be a friend?

No.

It has to be a **non-member** for the usual `cout << obj` syntax, but it doesn't necessarily have to be a friend.

If it can access the object's public interface, you can write:

```
ostream& operator<<(ostream& os, const Student& s) {  
    os << s.getAge();  
    return os;  
}
```

No `friend` needed.

Q8. Why return `ostream&` from `operator<<`?

To support chaining:

```
cout << a << b << c;
```

Advanced

Q9. Is a friend function a member function?

No.

It is a non-member function with special access privileges.

Q10. Does a friend function have a `this` pointer?

No.

Because it isn't a member function.

Q11. Is friendship inherited?

No.

Q12. Is friendship transitive?

No.

Q13. Is friendship mutual?

No.

Q14. Does `friend` violate encapsulation?

Not inherently. It provides explicitly controlled access, but excessive friendship can weaken encapsulation and increase coupling.

30-Second Interview Cheat Sheet

```
Operator Overloading
|
|— At least one operand → user-defined type
|— Cannot create new operators
|— Cannot change precedence
|— Cannot change associativity
|— Cannot change number of operands
|
|— Cannot overload:
    :: . .* ?: sizeof

Must be member:
    = [] () ->

<< / >>:
    Usually non-member
    Often friend
    because:
        cout << obj
        ↓
        operator<<(cout, obj)

friend:
    Non-member function/class
    with access to private/protected

friend function:
    no this pointer
    not inherited
    not transitive
    not mutual

operator<< return:
    ostream&
    → enables chaining
```

One thing to be especially careful about in interviews

Don't say:

"<< must be a friend function."

That's technically wrong.

The precise statement is:

`operator<<` is typically implemented as a non-member function because the left operand is `ostream`; it is often declared `friend` when it needs access to private/protected members.

That distinction is exactly the kind of detail an interviewer may use to separate a memorized answer from a solid C++ understanding.
